

Smart Tutor AI: Personalized Teaching Support

INFO 5731 - Computational Methods for Information Systems - Section 020

Spring 2025

College of Information

University of North Texas, Denton, TX

Christian M. Bridge

10918862

ChristianBridge@unt.edu

Litesh Perumalla

11760885

liteshperumalla@my.unt.edu

Pavan Kalyan Natukula

11751174

pavankalyannatukula@my.unt.edu

Sudikshya Devkota

11829101

sudikshyadevkota@my.unt.edu

Kusal Sai Ayinala

11742096

kusalsaiayinala@my.unt.edu

Harshitha Kalluru

11852561

harshithakalluru@my.unt.edu

Chinmai Kaveti

11746579

chinmaikaveti@my.unt.edu

Fardeen Ali Mohammed*

fardeenalimohammed@my.unt.edu

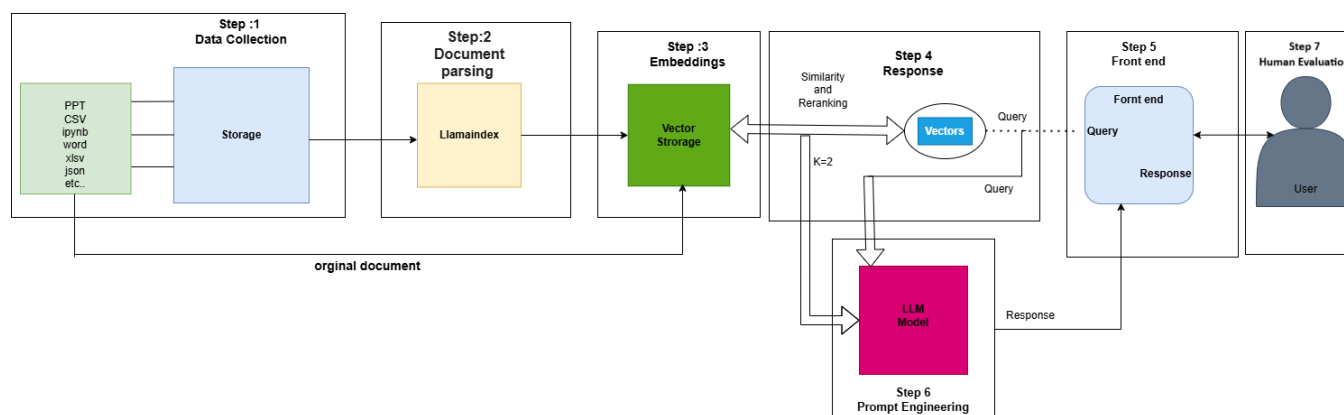


Figure 1. The pipeline of our Retrieval Augmented Generation-Large Language Model.

Abstract

Large Language Models (LLMs) have become ubiquitous due to free, user-facing models such as ChatGPT, Deepseek, and

*Teaching Assistant for the course and acted as team leader, creating the project idea, delegating assignments, and coordinating group progress.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. INFO 5731.020, Denton, TX, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN

<https://doi.org/>

Llama. These LLMs allow users to ask questions about many different topics, and can generate information like code in Python, answering questions about movies or TV shows, and creating workout plans; however, since these models are trained on massive sets of information, they are prone to hallucinations – misinformation generated without relying on actual source information. In order to improve the LLMs, an architecture known as Retrieval Augmented Generation (RAG) can be added to fine-tune LLMs using a set of source documents or data. For this project, we will be collecting documents from our Computational Methods for Information Systems class, using LlamaIndex to parse and chunk documents, embedding the documents with HuggingFace's all-MiniLM-L6-V2, storing them in a vector database using ChromaDB, and utilizing Llama 3.1 7b to create a Smart Tutor AI for Personalized Teaching Support. This architecture will allow the Llama model to use its previous training as well as new, course-specific documents to generate answers

for student support in the class. Lastly, these models can be deployed via a user-friendly chatbot front-end, where they can ask questions and receive a response. Since it is difficult to use common metrics, such as F1 scores, sensitivity, or accuracy, for model evaluation, we used a series of questions for human evaluation for fluency, coherence, relevancy, and factuality – human evaluations scored 1 to 5, 1 being poor performance and 5 being great performance, indicated an average model fluency of 4.85, coherence of 4.81, relevance of 4.32, and factuality of 3.97. Evidently AI was also used to evaluate context quality and response faithfulness with resulting model scores indicating that our model had 62% of responses not grounded in context, and 62% of retrieved context being irrelevant. We will then discuss the reasons for differing scores between metrics, possible improvements to increase model performance, and the future of RAG-LLM applications.

The code, data, analysis, and results can be accessed on GitHub at: <https://github.com/Fardeen210/AI-Driven-Personalized-Teaching-Support/tree/main>

CCS Concepts: • Computing methodologies → Natural language generation.

Keywords: RAG, LLM, LlamaIndex, Ollama, Llama 3.1, Parsing, Chunking, Vector Embeddings, AI, Smart Tutor, Teaching Support

ACM Reference Format:

Christian M. Bridge, Litesh Perumalla, Pavan Kalyan Natukula, Sudikshya Devkota, Kusal Sai Ayinala, Harshitha Kalluru, Chinmai Kaveti, and Fardeen Ali Mohammed. 2025. Smart Tutor AI: Personalized Teaching Support: INFO 5731 - Computational Methods for Information Systems - Section 020 Spring 2025 College of Information University of North Texas, Denton, TX. In . ACM, New York, NY, USA, 13 pages. <https://doi.org/>

1 Introduction

1.1 Background & Significance

In the present day, access to Large-Language Models (LLMs) has become ubiquitous with many commercially available models such as OpenAI's ChatGPT, DeepSeek, and Meta's Ollama – this means it has never been easier to have generative AI at your fingertips or to work on a LLM project. These models complete tasks such as translation, summarization, question answering, acting as personal trainers, creating meal plans, workout routines, and more.[6] A major strength of LLMs is that they are trained on massive datasets from various sources including websites, articles, books, blogs, social media, and more. However, this can be an issue as LLMs can excel at generating human-like responses, but the base AI models cannot keep up with constantly changing information in dynamic sectors, where using outdated information could lead to wrong or incomplete answers.[12] This also means that LLMs are trained on so much information

that it can "hallucinate" information or be inadequate for use in a domain other than what it was trained in, where fine-tuning using a given document set could further refine the model, improving performance.[14]

Large Language Models have a great potential for Natural Language Processing (NLP) tasks like Information Extraction but can face challenges on domain-specific questions creating hallucinations due to an overload of data, lack of contextual relevance, where as with Retrieval Augmented Generation (RAG) these hallucinations can be resolved.[2] LLMs can be expanded, or fine-tuned, to be more capable in a given domain by utilizing an input dataset as a set of vector embeddings, numerical representations of words which contain context and meaning, to pass source document chunks to the Retrieval-Augmented Generation (RAG) paradigm.[6] RAG allows researchers to create a model that has the power of a pre-trained LLM like Llama 3.1 which contains 8 billion parameters, and further enhance it using additional documents that were unavailable to it during initial training, such as files from our Computational Methods for Information Systems class. This empowers the LLM to perform better in new tasks such as acting as a Smart Tutor, or Personalized Teaching Support for our course. RAG-LLMs are not only effective as personalized teaching support, they can be used in other applications like businesses and sectors where technical documentation is constantly growing and high-productivity relies on efficient, effective information management systems and information retrieval methods.[17]

The significance of research like this is that LLMs can now be adapted to provide much better support to assist professors disseminating information to their class, as well as providing quick, automated responses when a student has questions. Our goal was to create a chatbot and evaluate performance using a series of questions to determine how accurately it could provide answers to our computational methods class documents, presentations, and Python programs. RAG-LLM models are also versatile, adaptable, and efficient to deploy and scale; although creating the initial pipeline was time consuming to learn and understand, RAG-LLMs once built can have new datasets added or swapped in the pipeline intake, can be used by multiple different users, and with proper embedding storage can efficiently process large amounts of documents and chunks.[2] The implications of being able to combine LLMs with RAG models mean that schools, businesses, and governments will be better able to train LLM models using domain-specific documents, which will help individuals find highly-specified information and have their questions answered automatically, instead of waiting for a human response, or needing to search through documentation. As mentioned previously, the usefulness of RAG-LLMs expands further than AI chatbots for classroom assistance, and can be beneficial to domains where highly specific or technical documentation is constantly changing,

and information management and retrieval can increase productivity and usefulness for individuals.

Some of the main difficulties we faced when working with RAG-LLMs were: data acquisition, text parsing, text pre-processing, chunking strategies, vector embedding models, and computational resources required to run larger LLM models. Khan et al. mention that in working with PDFs there can be many difficulties dealing with complex layouts, formats, embedded images, and other obstacles which hinder extraction of clean structured text data.[12] Another issue can be working with classified, trademarked, copyrighted, or proprietary information that is not publicly available. When handling information like this you must be cautious when using tools like HuggingFace embedding models and APIs, due to privacy, security, and compliance risk, as the files are being transmitted to outside of your secure environment. Also, depending on the model and need, you might not have vectors created that can properly distinguish differences between technical/industry specific words. For instance, using a generic embedding model will be okay for most language processing, however when handling information like medical terminology, some embedding models may not be equipped to create meaningful differences between key terms. This is because the similarity of the user query and chunks in the database are calculated using cosine similarity pictured in figure 2, which can determine how closely related a given query is to datasets stored in our database.

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

Figure 2. Calculation of cosine of the angle between two, 2-D vectors A and B.[29]

Lastly, while smaller LLM models exist like Llama 3.1 8b which can generally run on less powerful systems, we have found that installing locally can be a difficult endeavor without previous background knowledge, and that running larger models like Llama 3.3 70b would require much better systems and more time per response. While API connections to LLM models make it significantly easier to create a RAG-LLM and do not require an expensive computer to run, these models do require payment by tokens and can become quite expensive when running larger models. For example, Replicate, a service that hosts model runtimes, allows you to connect with an API and easily change models and parameters. Replicate hosts Llama 3.3-70b-instruct which allows individuals

to run massive models without needing the technical specifications locally, but costs \$1 for 1.5 million tokens input, and \$2.75 per 1 million tokens output.[24]

Our primary research questions that we answer in this project are: How can a RAG-LLM be created? What libraries, tools, and methods exist to create a RAG-LLM pipeline? What are the benefits of some of those tools or libraries versus others? Given we have chosen to use LlamaIndex, what tools are more compatible, or are easier to implement? Can our final model answer the human evaluation questions provided well? Is Evidently AI a good tool for evaluating RAG-LLM generated responses? And lastly, are there limitations of the final model, and how can those be expanded upon in the future?

1.2 Paper Overview

In our paper we will be reviewing related works, previous research, and work done in the RAG-LLM field that we can use to formulate our methodology and choices in deployment. The related works section is broken down into subsections of closely related topics that researchers have worked on like overall implementation of RAG models, the use of RAG-LLMs for intelligent tutoring, RAG-LLMs for general AI chatbots, methods for parsing data and creating vector embeddings, and mechanisms for evaluating RAG-LLMs and improving question answering.

In the Methodology, Data Collection & Cleaning, and Experiment & Data Analysis Plan sections we will be discussing the actual process for each of the steps in the RAG pipeline, such as collection of information, parsing data, pre-processing data and techniques, vector embedding models, trial and error of these steps, and the lessons learned from implementing different systems. Lastly, in the Results & Discussion section we will review the output of our model, Evidently AI's review of context quality and faithfulness, and human evaluations for fluency, coherence, relevance, and factuality. Lastly, we will discuss the conclusions we can draw from our results, what could be improved upon in future iterations, the future of the RAG-LLM architecture with ontology paradigms, and the assignment of individual project duties.

2 Related Work

2.1 RAG Models

Finardi et al. created a RAG model trained on the first Harry Potter book, *Harry Potter and the Philosopher's Stone*, and compared the results to OpenAI's ChatGPT; As we mentioned LLMs such as ChatGPT are pre-trained on vast amounts of data like information on Harry Potter, and was able to answer basic questions very accurately, but the model could only provide reasonable performance on more in-depth questions from the first novel, whereas the RAG-LLM could provide much more in-depth responses.[6]

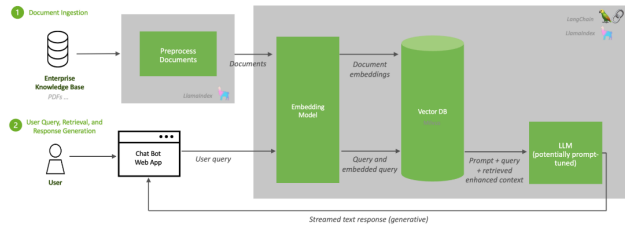


Figure 3. Overview of RAG pipeline components ingest and query flows.[32]

The paper, *Retrieval-Augmented Generation for Large Language Models: A Survey*, outline techniques for leveraging external knowledge and combating hallucination problems, out of date information, and diminished reasoning transparency[7]. It separates RAG into three evolving paradigms, namely, Naive, Advanced, and Modular RAG, and progressively assembles their sophistication in the retrieval and usage of information during generation. The survey is on the core components of RAG—retrieval, generation, and augmentation—and the optimization techniques, metrics of evaluation, and downstream applications like question answering, summarization, and code search. At the same time, it shows possible future directions: robustness, multimodal integration and integrations of RAG with fine tuning and reinforcement learning. These advancements make the RAG paradigm a promising technique in terms of increasing LLM performance in real world, NLP tasks like Information Extraction.

The paper *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*[15] introduces the framework RAG that improves language models using external document retrieval (using tools like DPR) in addition to pre-trained generation models (as demonstrated with BART). By using a hybrid approach, the model has the ability to retrieve up-to-date, factual information to use during generation for tasks such as open domain question answering, fact verification, and question generation. Compared to traditional models, RAG produces more accurate, diverse, and interpretable results, demonstrating the effectiveness of retrieving information and coupling it with language generation.

Retrieval Augmented Generation is a technique that adds all the generative AI of LLM with information retrieval and allowing tutor system to pull relevant system from external source into the LLM response.[27] By augmenting a generative AI's response with updated, user provided content the RAG can improve the relevance and accuracy of feedback to questions. By deploying RAG based tutors trained on course materials, the model can be personalized and generated explanations provide better context to students. Using external knowledge helps to minimize the hallucinations which are produced by LLMs, giving the incorrect response in a believable way – if you are not a subject matter expert and ask what you believe to be an expert a question,

the generated material may sound 100% accurate and could mislead users. Information retrieval when the document context is not found can confuse the LLM, so maintaining the information is difficult and the RAG should decide which information to retrieve. Lastly, deployment of a RAG-LLM in a User Interface is a complex task as it requires a persistent database, search index, AI model running, and logic to retrieve information.

2.2 Intelligent Tutoring System

The paper *TutorLLM: Customizing Learning Recommendations with Knowledge Tracing and Retrieval-Augmented Generation* introduces TutorLLM, an integration of LLMs, knowledge tracing (KT) and RAG to enhance education. To address personalization and accuracy issues, it used Multi-Features with Latent Relations BERT-based KT (MLFBK-KT) to track learning states, a scraper model for real-time content[16], and RAG-enhanced GPT-4 to produce responses.[14] A two-week study with 30 students showed a 5% quiz score improvement, 10% higher satisfaction, and 36% more engagement, showing its potential in AI-driven education. These researchers demonstrate the advanced potential of RAG-LLMs and how they can be synergized with other NLP tasks and methodologies.

Knowledge Graph - Retrieval Augmented Generation (KG-RAG), demonstrates an AI tutoring system's enhanced accuracy, empirical validation, and personalization by integrating KG with RAG, unlike traditional RAG, which solves the problem of information hallucination and retrieves isolated text snippets[9], KG-RAG structures concepts logically, traversing the knowledge graph to add related concepts. It shows that using DeepSeek-V3, a cost-effective model, ensures better reasoning and shows a 90.2% accuracy on the MATH benchmark.[17] A study with 76 students showed a 35% higher assessment score, demonstrating superiority in precision, and engagement.

The paper *Feature-Aware Knowledge Tracing for Generation of Concept-Knowledge Reports* introduces a novel approach to enhancing personalized learning experiences. By integrating feature-aware KT, the system can generate detailed reports on students' understanding of specific concepts, allowing educators to tailor instruction more effectively. This method not only tracks student performance but also considers various features that influence learning, leading to more accurate assessments and targeted interventions.[10] The implementation of such a system demonstrates significant improvements in identifying knowledge gaps and promoting efficient learning pathways.

The paper *Advancing Generative Intelligent Tutoring Systems with GPT-4: Design, Evaluation, and a Modular Framework for Future Learning Platforms*. Intelligent Tutoring Systems are computer based learning systems that utilize Artificial Intelligence for personalized learning experience based on the needs of each user. [18] These are integrated with

student model and feedback system by implementing this techniques leads to a better learning environment for tutoring efficiency. But traditional ITS are mostly based on rule based and pre defined content which is not diverse. This limitation made the creation of Intelligent Tutoring system which has flexibility and acts as a Human Tutor and adapts to the new content. The initial results of ITS showed the potential to interact students and offering support with academic problems. But there are still challenges that needs to be addressed like frameworks should be made such that it is easier to integrate to the tutoring systems. And the computational costs for running the ITS and validation of content to prevent biased feedback and hallucination. And aligning generative AI with curriculum objectives. While the current ITS are powerful enough, there is still development needed in this field that needs to complete achievement.

2.3 Integrating LLMs and RAG for educational AI chat bots

The paper *Integrating a LLaMa-based Chatbot with Augmented Retrieval Generation as a Complementary Educational Tool for High School and College Students* provides research about RAG-LLMs as a tool to improve the learning experience and educational strategies of students. Dario et al. developed a RAG based chatbot using LLaMa-7B-chat-model and integrated with a mathematics database, comprised of a pre-calculus textbook, instructor feedback, audiovisual resources, and Pinecone API for information retrieval.[3] Their research also compares the responses from the different AI chatbots to prove the efficiency of the proposed model in the education industry. The findings convey that AI driven educational AI assistants can help students learn and obtain good knowledge acquisition.

While our focus is on providing a model which is trained on course documents, and can act as a teaching assistant, there are other areas in education settings which could benefit from RAG models. Lekan and Pardos in their paper discuss the use-case of a LLM to provide undergraduate students with Major recommendations, like an Academic Advisor.[13] While they did not use a RAG model to further refine the LLM based on university specific documents or academic advisors, an expansion of their idea is that if they were to obtain such information, they could further fine-tune the model to match the results obtained from advising professionals.

In *Utilizing Large Language Models for Question Answering in Task-Oriented Dialogues* Sticha introduces ChatPLT, an AI-based Intelligent Tutoring System (ITS) designed to support physics education in higher learning institutions. Utilizing LLMs, RAG with Ollama for content processing, and LlamaIndex for document classification and vector-based retrieval, integrated into a user-friendly Streamlit interface the system provides controlled responses by referencing pre-selected documents, ensuring accuracy while minimizing misinformation.[28] The AI tutor aims to assist students in

identifying and resolving misconceptions in Physics (Physical Techniques I) while enabling predictive learning analysis based on student interactions. Initial testing, which involved 40 questions, showed 77.5% correct responses, though 20% contained missing or incomplete information[28], and 2.5% were incorrect, highlighting the need for human supervision. Despite its potential, the system's limited scope in Physics and early-stage development restrict its broader application, though ongoing improvements aim to enhance its accuracy and adaptability.

The study performed by Naya-Foracano et al. in *ChatPLT: An Intelligent Tutoring System for Teaching Physics in Higher Education*, examines the impact of a GPT-4o mini-based AI tutor on psychology students learning R programming at the University of Tübingen. The AI tutor was integrated into a web-based interface with a custom system prompt designed for interactive guidance, while video tutorials were used as a comparative learning method. Students were divided into three groups: those using the AI tutor, those following video tutorials, and those using both methods. The AI tutor provided hints instead of direct answers, encouraging self-directed learning[22], while Likert-scale evaluations measured both performance and student satisfaction. However, results indicated no significant differences in learning outcomes across the groups, partly due to the small sample size, which limited statistical power. While students expressed a preference for AI tutors, this did not translate into measurable efficiency improvements. Additionally, concerns were raised about potential over-reliance on AI tutors and inconsistent engagement levels with the assigned materials.

A paper titled *Using Retrieval Augmented Generative AI Chatbots to Support and Enhance Chemistry Education* explores the integration of Retrieval Augmented Generation (RAG) models into chemistry education. RAG models combine the information retrieval systems and generative AI capabilities to let chatbots go to external knowledge sources and have accurate responses too. The authors discuss the use of these sophisticated chatbots in interactive playing the role of a teaching tool equipped with giving students immediate feedback, personal explanations and responses to challenging chemistry questions. They also talk about the possible matters of academic integrity preservation, bias control, and accuracy of information that could be fetched by you.[4] The paper concludes that RAG based chatbots can have a great impact on improving chemistry education through providing dynamic and responsive support when appropriately used by students.

2.4 Parsing Data and Vector Embeddings

In *Parsers and Parsing Approaches: Classification and State of the Art* the authors cover all possible parsing techniques in computational linguistics in detail. They describe parsers as two types: syntactic parsers, consisting of constituency and dependency parsing, and semantic parsers, shallow and

deep semantic parsing.[23] Moreover, they address different parsing algorithms including top-down and bottom-up parsing, highlight the benefits of dependency parsing for the free word order languages, discuss problems such as redundancy in sentence parsing and unstructured data, and ascertain the suitability of parsing for specific natural language processing.[23]

In *Word2vec, node2vec, graph2vec, X2vec: Towards a Theory of Vector Embeddings of Structured Data*, the authors examine vector embeddings for structured data such as graphs and relational structures; Although embeddings are used ubiquitously in machine learning, little attention has been paid to its theoretically most simple and basic properties. Grohe reviews the expressivity and efficiency of existing techniques, in particular Graph Neural Networks (GNNs) and knowledge graph embeddings, and node embeddings where the design of homomorphism-based methods like Weisfeiler-Leman as a central tool for understanding embeddings and comparing graphs.[8] Grohe then takes the first steps towards connecting these empirical pieces to the theoretical insights and provides directions for future work that embeds quality, computes computing efficiency, and the relational structure of proxies of higher arity.

In chapter titled "Vector Semantics and Embeddings," introduces the concept of converting the test words in to numerical representations using vector representations known as vector embeddings which is a core for NLP. It describes how earlier linguistic theories developed in to modern vector based approaches for word representation. The authors the explained the key concepts such as term-document matrix, TF-IDF weighting, and Point Wise Mutual Information.[1] They explained how these methods can be used to measure the word similarity for a given query using cosine similarity from the word embeddings.

2.5 Improving Faithfulness in AI-Driven Question Answering

In *The Impact of an AI-Based Tutor Using GPT-4o Mini on Learning R Programming among Psychology Students at the University of Tübingen*, researchers explore the use of LLMs, including GPT-3, GPT-4, and BART, for question-answering in task-oriented dialogues. Von Dielingen et al. employ the RAG framework and LlamaIndex for efficient document retrieval, integrating the ReAct framework to enhance reasoning and tool usage, and focus on improving faithfulness by reducing hallucinations using a re-ranking technique for better document selection, and a self-reflection mechanism to ground responses in retrieved content.[30] The Multi-Doc2Dial dataset is used to test the models, incorporating few-shot prompting and a novel self-evaluation framework with GPT-4.[30] While the approach improves response accuracy and linguistic quality, limitations include occasional hallucinations, evaluation biases favoring longer responses, high computational costs, and dataset constraints that limit

generalization beyond task-oriented dialogues. Traditional methods like fine-tuning and prompt engineering struggle to adapt language models for real-world K-12 educational settings because they rely on surface-level patterns and fail to capture the latent reasoning processes that expert educators use. However, the Bridge method used in Tutor CoPilot captures expert decision-making through think-aloud protocols, enabling more contextually appropriate suggestions. [31] Prior approaches to evaluating AI tutors have lacked a unified framework, relying on disparate and often incomplete taxonomies. To address this,[21] proposed a comprehensive evaluation taxonomy grounded in learning sciences principles, comprising eight pedagogical dimensions specifically designed for assessing AI tutors in the context of student mistake remediation.

3 Methodology

The development of Smart Tutor AI follows a structured methodology consisting of seven key phases: data collection, document parsing, embedding creation & storage, response management, building a front end, prompt engineering, and evaluation. First, data was collected from our classroom management system, UNT Canvas, to provide a robust knowledge base for our RAG-LLM. Next, the data was cleaned removing anything that may take away from the semantic meaning or similarity of words, such as special characters or irrelevant text. After cleaning, the text was chunked into sections to allow for smaller and more compact groups of text, this makes searching text for similarity easier. With the chunks created, they were stored in a vector embedding database which transformed each chunk of documents into a series of numbers which can be used for similarity comparisons. With everything cleaned and stored in the database, we were able to connect a LLM to the pipeline, initially using Replicate API for testing, but finally by installing the Llama 3.1 8b locally and creating a virtual environment. Next was prompt engineering, a very crucial step to ensure the model is relying solely on the newly provided vector embedding database context, instead of previously learned knowledge to limit hallucinations, and to return data as a specific agent type, or with a required format. With these steps completed, a simple local User Interface (UI) website was created using Streamlit, which allowed users to enter a query to the model. The query was then transformed using the same vector embedding model used to created the vector embeddings in storage, and returned K similar vectors in the database; For our model, we returned a top K value of 3. Once the top K vectors' context was returned, it was fed into the LLM model and output as generated text where the LLM tried to simplify and structure as an output using the engineered prompt. Lastly, after testing with different chunking sizes, K values, prompts, embedding models, ensuring sufficient data sources, good pre-processing strategies for data, and

LLM model choice, we can evaluate it using our evaluation questions to judge the faithfulness and context quality using Evidently AI, and the fluency, coherence, relevance, and factuality with human evaluation.

3.1 Data Collection & Cleaning

The scope of our project is to act as a Smart Tutor AI for our course subject, Computational Methods for Information Systems, so we gathered learning materials like PowerPoint presentations (.ppt, .pptx), python files (.py, .ipynb), and documents (.pdf, .txt, .docx) provided by our instructor, Dr. Haihua Chen, from our online course repository, UNT Canvas. These data sources contained information about the course like class presentations on coding with sample code screenshots, python notebooks with text and code blocks, research papers, journals, and articles. The documents were stored in a GitHub repository for easier group consolidation and retrieval.

Next, these materials were parsed and organized using LlamaIndex, which provides an all-in-one document parser called SimpleDirectoryReader. The main benefit of SimpleDirectoryReader is that it is compatible with many different types of file extension by default, and does not require initialization or separation of input files; This allowed efficient indexing and retrieval by organizing the data into a format that can be interpreted computationally [20]. However, we did run into issues parsing such a variety of documents, which made data cleaning very difficult. For example, PowerPoint files like .ppt were not initially compatible, although documentation indicated that they were, so conversion of .ppt files to .pptx was required. The second major issue was that clip art, background formatting, reoccurring page footers, and screenshots all would cause inappropriate, irrelevant data injections into parsed text – this could be lists of items being interrupted by background format, images being a long unusable file name, and page structure causing confusion in relation to text, sometimes mixing text sections up creating inaccurate and illiterate sentences. This issue could occur on PDF documents, powerpoints, and python notebooks, so in an attempt to control the text extraction, we had copy and pasted text from some of the data sources into text documents (.txt) to avoid these unforeseen parsing errors, and control sentence structure.

Next, the parsed documents were cleaned and preprocessed to remove punctuation, special characters, and multiple spaces in non-code files (.pdf, .txt, .pptx, etc.), however code files like .ipynb and .py were untouched as punctuation, multiple spaces, and special characters could have meaning in code.

4 Experiment & Data Analysis Plan

Next, we experimented with Llama Index's SentenceSplitter parameters which control the chunk size of text data before

being passed to vector embeddings. Initially, we used a chunk size of 100 with an overlap of 10, then we experimented with a chunk size of 128 and overlap 2, however based on the output and preliminary results, we eventually parsed the documents into chunks of size 512, with a chunk overlap of 10. The methods for chunking can be an entire research paper on it's own as there are many different methods to obtain chunks (character splitting, semantic splitting, recursive splitting), and chunk sizes which can impact the context being fed into a model; With larger chunks being harder to query, but more likely to encapsulate information, and smaller chunks being easier to query but highly likely to exclude related information. With the chunks created, we were ready to use Llama Index's IngestionPipeline to process the chunks into embeddings.

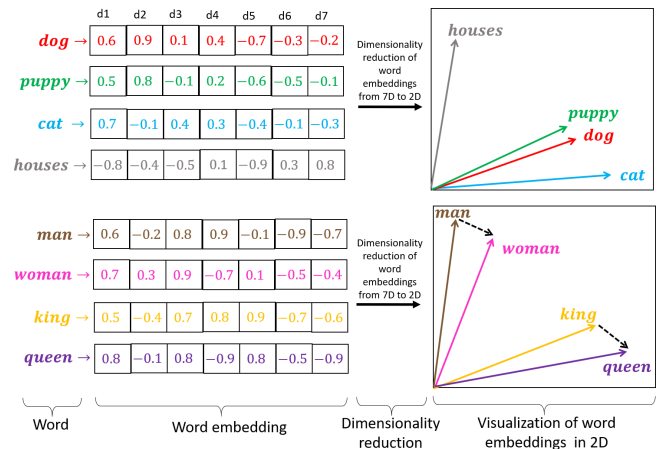


Figure 4. Word embeddings map words in a corpus of text to vector space.[25]

We had to fine-tune the ingestion and parsing of documents for creating embeddings and using different transformer models to embed and store information in our vector database. Initially after trying one transformer type, the team decided to switch to using the Hugging Face transformer as it is more efficient and easier to integrate with our model. We also tried using Qdrant Vector store, but after running into issues with setup and deployment, decided to use ChromaDB because it is lighter and easier to store the vector embeddings. During this process, the team faced many errors regarding compatible issues with dependencies, as well as initializing ChromaDB and ingesting embeddings to store. Therefore, our final model utilized IngestionPipeline to create vector embeddings using the HuggingFace embedding model sentence-transformers/all-MiniLM-L6-v2, which captures semantic relationships in sentences, facilitating efficient information retrieval stored in our specialized vector database. This is another area that required experimentation as there are many different embedding models, some of which are

specialized for use on highly technical datasets like medical terminology/text, however we chose this specific one due to accessibility, relative small model size, and available documentation on HuggingFace for Sentence-Transformers.

Next, we integrated the vector embedding database with a Llama Index RAG Query Engine, which means that when a student submits a query using the front-end chatbot, the model will convert the query into vector embeddings and search the database for closely related embeddings from the database, which will be fed into our LLM model to generate a tailored response. Although this appears to be simple, there are many different types of query engine and chat engines provided by Llamaindex as well as parameters for each, so users can create basic or advanced responses using tools such as Chat Engine, RAG Query Engine, Retriever, and Response Synthesizer. These methods take reading a lot of documentation, as well as trial and error to understand which is best for our usecase, how to personalize and improve upon the model following their documentation, and how to create an effectively engineered prompt for response output[19]. For our purposes we stuck with the basic query engine for question and answer. Lastly, we engineered a prompt to ensure the model would act as a teaching assistant, only use context information and not prior knowledge. Some additional considerations were made for validation of results as well like including the document the context was being selected from.

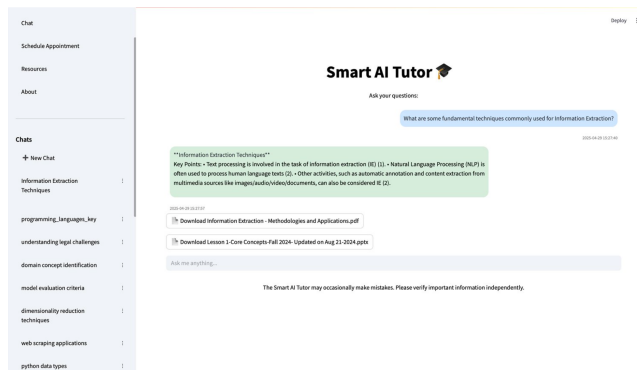


Figure 5. Our Streamlit User Interface

4.1 Human Evaluation & Evidently AI

Finally, to conduct a proper human evaluation on the effectiveness of Smart Tutor AI, we systematically developed a set of questions by analyzing each module's course content input files, and created sixty-two evaluation questions. Our approach involved reviewing all available material across various file types to derive both general and specific questions that would test the model's comprehension and ability to support students. Some of the answers used for evaluation were directly copy and pasted from the module content to

test factual retrieval, while others were reorganized without being reproduced exactly. This allowed us to test not only the accuracy of information, but also the contextual understanding demonstrated by the model.

Examples of the human evaluation questions are as follows (a full list of questions and responses can be found in the github evaluation folder):

1. What are the commonly used data types in python in week 2?
2. What are the web scrapping applications mentioned in week 4 module?
3. What is POS tagging which is mentioned in week 6 module?
4. How can we use Python for feature extraction which is mentioned in week 6 module?
5. What are the top programming languages in 2025 as per week 1 module?

We then had multiple people give evaluations for each question and response pair on a scale from 1 to 5, 1 being poor performance and 5 being great performance, for fluency, coherence, relevance, and factuality. Fluency being the naturalness and grammatical correctness of generated text, coherence the logical flow and consistency, factuality the accuracy of the response, and relevance the appropriateness/relatability to the question.[11] We also used a library named Evidently AI to evaluate the context quality and faithfulness of answers generated to a question given the initial context from input documents, where faithfulness is how true the generated text stays to the retrieved context, and context quality is the ability of retrieved context to answer the question. [5]

5 Results & Discussion

5.1 Human Evaluation Results

We can see in the human evaluations that overall the model did a good job at producing fluent and coherent responses, with an average user rating across all questions of 4.69 and 4.52 respectively, which is definitely displays how well current LLMs are able to generate cohesive text. However, we can see while the model creates answers that appear to be human-like and logical they can often only be highly related to the question asked, with the factuality being on average a little lower, with scores of 3.98 and 3.85 respectively. We can also see that the variability of fluency and coherence may vary but is not often less than a 3.5, whereas the variability for relevance and factuality can range massively from question to question.

We can also see when looking at the top 10 and bottom 10 questions based on factuality that fluency and coherence do not always mean relevant or factual.

For instance, when we examine question #29 deemed by human evaluation to be the lowest factual response provided,

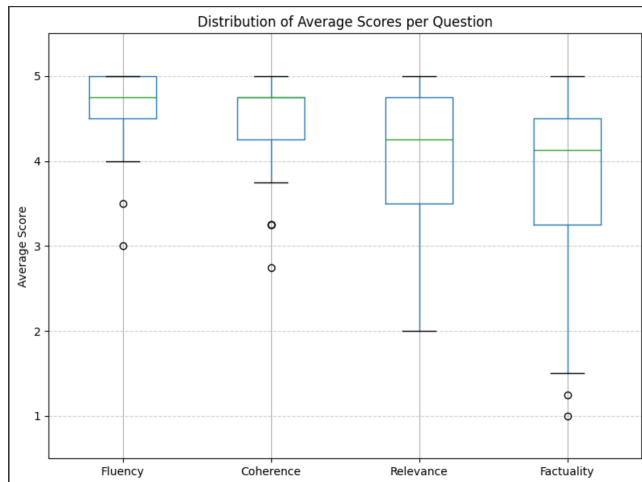


Figure 6. Boxplot for the average human evaluation scores of model responses per question.

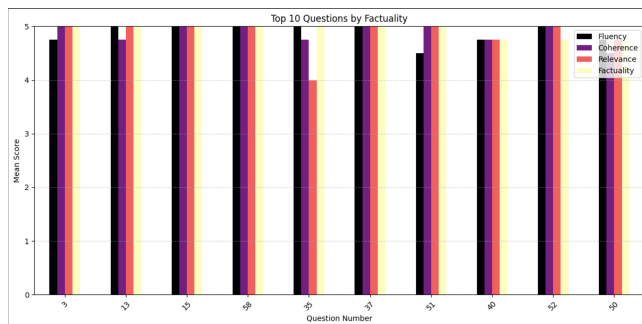


Figure 7. The top 10 questions based on average factuality score.

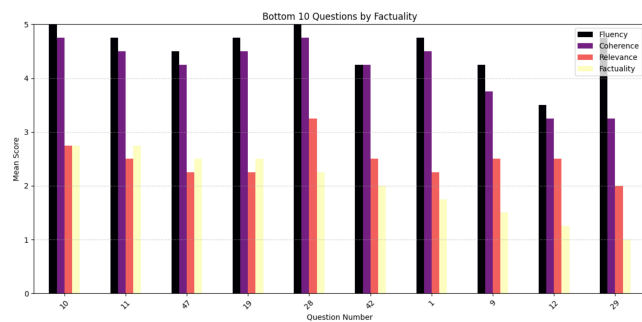


Figure 8. The bottom 10 questions based on average factuality score.

we can see the fluency and coherence are high, and even the relevance is moderate, but the actual answer itself is wrong.

Question: What are the five categories into which the WordNet lexicon is divided according to week 7 module?

Response: The WordNet lexicon is divided into five categories:

1. **Synsets**: A set of synonyms that convey nearly the same meaning.
2. **Hypnyms**: These are more general terms and concepts.
3. **Hyponyms**: These are more specific terms and concepts, which are subcategories under hypnyms.
4. **Holonyms**: This refers to a relationship between words where one word is a part of another word.
5. **Meronyms**: This term denotes the opposite of holonymy in WordNet – where one concept is contained within another.

To understand this, consider an example: If we have a synset for the word "car", then its hyponyms would include words like 'sedan', 'trunk', and so on.

While it is true that WordNet contains the aforementioned lexical relations, the provided dataset document *Lesson Six-WordNet PowerPoint* indicates "division of the lexicon into five categories: Nouns, verbs, adjectives, adverbs, function words"; Therefore, it would be extremely difficult for a prospective student to understand that they are being mislead, because the answer is given truthfully and authoritatively, but misses the main context of the global division of the lexicon.

5.2 Evident AI Results

Evidently AI defines context quality as the model that provides sufficient enough information to answer the question; Faithfulness is a measure of similarity to the original context without generating hallucinations or unsupported claims.[5] The Evidently AI results are provided below in figure 2, which indicates our answers in the graph as percentages.

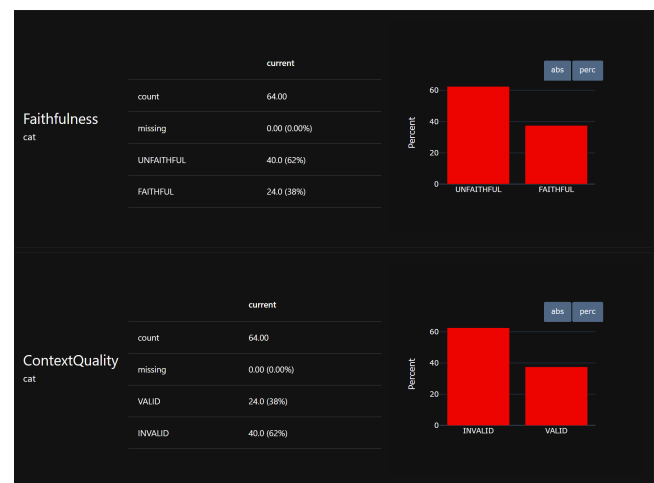


Figure 9. The Evidently AI Evaluation of Context Quality and Faithfulness of Our Models Responses.

From the Evidently AI results we can conclude that the answers which were given by the RAG-LLM model appear to be lacking context quality with 40 of 64 questions, 62% being considered as not answering the question. The model is also highly unfaithful with 62%, 40 of 64 questions, responded to in a way that may highly deviate, hallucinate, or answer with claims not supported in the original context.

While these evaluations appear to be extremely negative, when looking at the output from the model for each question, and our aforementioned human evaluations, it appears to be doing well at giving answers, but may be slightly off-topic or not based on data source context. For example, question one deemed as unfaithful, but valid context asks "What are the commonly used data types in python in week 2?" to which the model replies:

"Assistant: In Python, you'll work with various data types. In Week 2, we're focusing on lists and how to manipulate them efficiently using different operations such as indexing, slicing, insertion, and deletion. Lists are crucial for storing collections of items, which can be numbers, strings, or even other lists.

Here's a simple example to illustrate this:

“python

Create a list with some values

fruits = ['apple', 'banana', 'cherry']

Accessing elements in the list using indexing (0-based)

print(fruits[0]) # Output: apple

Slicing to get a subset of the list

print(fruits[1:]) # Output: ['banana', 'cherry'] “

This example showcases basic usage, but keep in mind that there are many more operations and features you can use with lists."

Evidently AI Responses: Unfaithful – The response describes working with lists in Python, which is not mentioned or supported by the provided source. The source focuses on various topics related to Python, including syntax and basic concepts, but does not cover lists specifically. Therefore, the information in the response contradicts the information in the source.

Context Quality Valid – The text includes a section that specifically mentions 'Commonly used python data types' and goes on to describe Python Numbers, which encompass int, float, and complex types. These directly relate to the question about commonly used data types in Python, making the text sufficient to answer the question effectively.

For comparison, human evaluations on average ranked the response as highly fluent (4.75), highly coherent (4.5), slightly relevant (2.25), and low factuality (1.75).

We can concur with the Evidently AI results that the model response focuses on working with lists and indexing, rather than the data types one would expect from the context like int, float, or complex, etc. In the next section we will review limitations of our model that could impact these scores, but first we will review an example Evidently AI deemed as faithful and valid, question nine, "What are the steps for web scrapping as per week 4 module?":

To extract data from the web using web scraping, we can break it down into several key steps.

Firstly, we need to identify the source of the data and determine which specific information is required. This could involve analyzing websites, online forms, or other digital sources.

Next, we must figure out how to access this data programmatically. This may involve using web development tools, such as HTML and CSS selectors, or learning programming languages like Python.

Once we have a way to access the data, we need to write code to extract it from the website and store it in a format that's easy to analyze.

Finally, after extracting the data, we should review it for accuracy and completeness to ensure it meets our needs.

Evidently AI Response: Faithful – The text describes steps for web scraping that align with the source's description of collecting text data and using web development tools and programming languages like Python. It covers identifying data sources, accessing data, writing code for extraction, and reviewing data, which corresponds to the comprehensive overview of web scraping techniques provided in the source.

Context Quality Valid - The text includes clear steps for web scraping: 'Download the web page', 'Parse the page', 'Locate and download the text information we need', and 'Save the text collected'. While additional information about tools and libraries is present, the essential steps are explicitly stated.

For comparison, human evaluation ranked the response as highly fluent (4.25), moderately coherent (3.75), slightly relevant (2.5), and low factuality (1.5). This highlights the subjectivity of human review and knowledge basis, where the generated response may highlight the overview of steps required as listed on the *Less Four - Python Web Scraping* PowerPoint (Download the web-page, Parse the page, Locate and download the text information we need, Save the text collected), human respondents when receiving the answer may be looking for something more in-depth like code provided on how to programatically access and parse data.

With the answer for question nine we can find that the faithfulness is good because the generated response closely matches the input context from our data sources, and the context quality is good because it accurately describes the steps for webscrapping.

6 Conclusion & Limitations

While our model does not have the highest faithfulness and context quality scores, this could be due to a variety of reasons. First and primarily, we cannot leave LLM evaluation up to other LLMs, because they are prone to issues just as the one being evaluated. For instance, some of the responses of the model provide an answer, and then break it down into easy-to-understand analogies, which are not present in the original context, which could appear to be highly hallucinatory. Another issue can be determining context quality when dealing with messy data because the context extracted could answer the question, but might be full of extraneous information like page numbers, authors, or missing information that was removed due to being an image that could severely impact the score.

When we compare these results from Evidently AI to human evaluations, we find that human evaluators are more forgiving of the responses and appear to rate coherence, fluency, factuality, and relevance higher. This too can be a limitation of human evaluations though, because it relies on the subject evaluating the response to have access to the correct answer or know the subject matter well enough that they can accurately determine if the response is true, versus sounding true enough. Relying on human evaluation also means scoring is subjective, where one person believes the RAG-LLM answering a list is a common type of python data type is a relatively acceptable answer, whereas another student would rate the model very poorly because they expect the response to be integer, float, string, boolean, etc. Lastly, the process is extremely time consuming and getting individuals to focus enough to be able to evaluate sixty-two questions, for four different metrics, totaling 250 scores can have a chance where individuals avoid thorough evaluation by rushing, or replying dishonestly.

Adding to the aforementioned limitations of model evaluation, building RAG-LLMs is an iterative process and may take several attempts to create a good model since many small tweaks can cause major changes in performance. For us, one improvement would be reviewing the input data especially from formats like .pptx which may be including meta data not specific to the course content, or deleting pertinent information that is only stored in screenshots on slides. By adding something like Optical Character Recognition (OCR) we may be able to derive information from images like screenshots of code, or architecture diagrams. Also, the computational requirements to run larger and better LLMs is a limitation,

which access to better funding or better computational resources could improve, and therefore enable the RAG-LLM to use better models like Llama 3.3 70b. Our model is also only hosted locally which severely restricts the ability for individuals outside of the group to access, query, and evaluate, where if it was hosted and distributed online, we could get more human evaluations and test questions outside of the created sixty-two. Another improvement would be including more supplemental material in our data sources, such as videos (.mp4 and youtube), audio files, entire github repositories, and web-scraping links. Other steps could be looking at different embedding models, which might create more semantic difference for words specific to working with NLP coding and computational method terminology, different text splitting methods like SemanticSplitter or RecursiveSplitting, and experimenting with different chunk sizes and overlap for vector embedding storage. Lastly, different query engines for getting a response from the vector embeddings and LLM model that could incorporate chat history, or looking at various parameters like temperature, and different methods of prompt engineering, which may limit hallucinations and therefore unfaithful or poor context quality responses.

6.1 The future RAG-LLMs

The future of RAG-LLMs is likely to change a lot with the help of ontology paradigms. A Microsoft Research team Sharma et al. discovered that Ontology-Grounded Retrieval-Augmented Generation (OG-RAG) can enhance LLM-generated responses by anchoring retrieval processes in domain-specific ontologies; This architecture looks to improve on RAG structures by incorporating documents and domain-expert created ontologies, structured representations between key entities and their relationships within a domain, to adapt LLM responses to complex and evolving information landscapes.[26] The benefit of adding these to RAG-LLM framework is that it goes beyond the basic general-purpose embeddings and ad-hoc context generation without grounded domain expertise by using hypergraph retrieval to adapt the LLMs, where a set of relevant hypernodes is identified by the query, then relevant context is formed as the set of hyperedges that minimally cover the relevant hypernodes, solving for the optimization problem to establish a limited number of hyperedges representing facts relevant to the query, grouping the related facts, and passing the compact and comprehensive context to the LLM response.[26] Basically, using a structured understanding of how domain-specific data interacts within a sector, the LLMs will be better able to understand hierarchy and relationships that are only intrinsic in that domain, and can therefore optimize the generated response due to the created ontology.

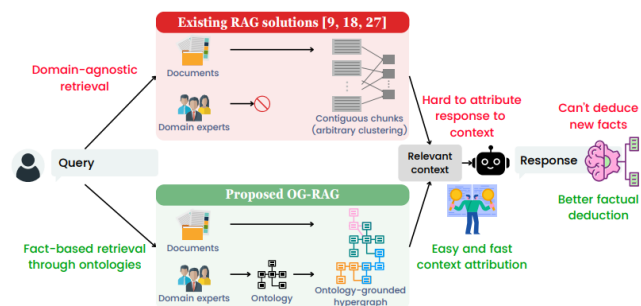


Figure 10. Image by Microsoft Research. Comparison of their proposed OG-RAG with current RAG-LLMs

7 Author Contributions

There were multiple sections of the RAG pipeline that were broken down into separate teams. First, everyone was assigned a module from the Canvas online course website to download the files and add them to the Github repository. All group members also had to further manually pre-process some of the dataset by copying text directly from the source material, and pasting into a .txt file for easier parsing, and noise reduction.

Initially everyone had tested using SimpleDirectoryReader to parse documents, however Litesh, Pavan Kalyan, and Kusal Sai worked on the final code to ingest the data sources within each module and parse. After the documents have been consolidated into similar formats, they worked on the vector embeddings and storage initially using Qdrant, then HuggingFace and ChromaDB. This also required working on the data parsing and chunking strategies. Lastly, the group implemented the model locally, engineered the prompt, and created the streamlit user interface.

Christian and Chinmai researched how to connect the vector embedding database to an LLM model. Initially, Christian was able to create a basic RAG-LLM and demonstrate the LlamaIndex chat engine options using Replicate API, but the group wanted to move to a local model to avoid API costs. Chinmai and Christian were able to research model installation and querying the RAG-LLM system, but due to issues synchronizing with the github repository and setting up the local virtual environment, the research and code was passed onto the aforementioned team to integrate with their developed applications.

Harshitha and Sudikshya worked on the model evaluation questions which were overseen by Fardeen Ali, and were used for Evidently AI and human evaluations of the final model. Questions that were created also have the relative module where the answer can be found which will make human evaluation of model generated answers easier.

Litesh created the google colab python file to use Evidently AI for evaluating the Faithfulness and Context Quality of the model. Christian lead the team's human evaluation effort

with Litesh, Pavan Kalyan, and Chinmai also evaluating all 63 questions manually for fluency, coherence, relevance, and factuality.

Christian was the primary author and editor for all research paper submissions and PowerPoint presentations. All group members were responsible for finding at least 2 related works to include in the paper, and to check the accuracy of what was written about the section they were working on.

The project was overseen and managed by Fardeen Ali, he assigned work to individuals and grouped them to work together and collaborate. He would also host weekly meetings for the team to share progress and managed the top-level GitHub repository's push and pull requests. Lastly, he would recommend architecture, RAG-LLM creation libraries, documentation, and assist on project requirements as needed by group members.

References

- [1] 2025. *Speech and Language Processing* (2nd. ed.). Pearson, Upper Saddle River, NJ. <https://web.stanford.edu/~jurafsky/slp3/6.pdf>
- [2] Muhammad Arslan, Saba Munawar, and Christophe Cruz. 2024. Business insights using RAG-LLMs: a review and case study. *Journal of Decision Systems* (2024). <https://doi.org/10.1080/12460125.2024.2410040>
- [3] Fonseca-Delgado R. Reyes-Chacón Iván Vizcaino Paulina Morocho-Cayamcela Manuel Eugenio Cabezas, Dario. 2024. Integrating a LLaMa-based Chatbot with Augmented Retrieval Generation as a Complementary Educational Tool for High School and College Students. (2024). https://www.researchgate.net/publication/382226023_Integrating_a_LLaMa-based_Chatbot_with_Augmented_Retrieval_Generation_as_a_Complementary_Educational_Tool_for_High_School_and_College_Students
- [4] Melanie M. Cooper and Michael W. Klymkowsky. 2024. Let Us Not Squander the Affordances of LLMs for the Sake of Expedience: Using Retrieval Augmented Generative AI Chatbots to Support and Evaluate Student Reasoning. *Journal of Chemical Education* 101 (2024). <https://pubs.acs.org/doi/10.1021/acs.jchemed.4c00765>
- [5] EvidentlyAI. 2025. Metrics: All Descriptors. https://docs.evidentlyai.com/metrics/all_descriptors#rag Last accessed 15 April 2025.
- [6] Paulo Finardi, Leonardo Avila Marcos Piau, Rodrigo Castaldoni Pablo Costa, Pedro Gengo Vinicius Carida, and Celio Larcher. 2024. The Chronicles of RAG: The Retriever, The Chunk and the Generator. (2024), 1–16. <https://arxiv.org/pdf/2401.07883>
- [7] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2024. Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv preprint arXiv:2312.10997* (2024). <https://arxiv.org/abs/2312.10997>
- [8] Martin Grohe. 2020. Word2vec, node2vec, graph2vec, X2vec: Towards a Theory of Vector Embeddings of Structured Data. *arXiv preprint arXiv:2003.12590* (2020). <https://arxiv.org/abs/2003.12590>
- [9] L. Guo et al. 2021. Evolution and Trends in Intelligent Tutoring Systems Research: A Multidisciplinary and Scientometric View. *Asia Pacific Education Review* 22 (2021), 441–461. <https://arxiv.org/abs/2412.19437>
- [10] Mithun Haridas, Nirmala Vasudevan, Surya Gayathry, Georg Gutjahr, Raghu Raman, and Prema Nedungadi. 2019. Feature-Aware Knowledge Tracing for Generation of Concept-Knowledge Reports in an Intelligent Tutoring System. In *IEEE Tenth International Conference on Technology for Education (T4E)*. IEEE, 142–145. doi:10.1109/T4E.2019.00-34
- [11] Stanislav Karzhev. 2024. LLM Evaluation: Metrics, Methodologies, Best Practices. <https://www.datacamp.com/blog/llm-evaluation> Last accessed 1 May 2025.

- [12] Ayman Asad Khan, Md Toufique Hasan, Kai Kristian Kemell, Jussi Rasku, and Pekka Abrahamsson. 2024. Developing Retrieval Augmented Generation (RAG) based LLM systems from PDFs: An Experience Report. (2024), 1–36. <https://arxiv.org/abs/2410.15944>
- [13] Kasra Lekan and Zachary A. Pardos. 2024. AI-Augmented Advising: A Comparative Study of GPT-4 and Advisor-based Major Recommendations. In *Proceedings of the 2024 AAAI Conference on Artificial Intelligence (Proceedings of Machine Learning Research, Vol. 257)*, Muktha Ananda, Debshila Basu Malick, Jill Burstein, Lydia T. Liu, Zitao Liu, James Sharpnack, Zichao Wang, and Serena Wang (Eds.). PMLR, 85–96. <https://proceedings.mlr.press/v257/lekan24a.html>
- [14] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 793, 16 pages.
- [15] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *arXiv preprint arXiv:2005.11401* (2020). <https://arxiv.org/abs/2005.11401>
- [16] Zhaoxing Li, Mark Jacobsen, Lei Shi, Yunzhan Zhou, and Jindi Wang. 2023. Broader and Deeper: A Multi-Features with Latent Relations BERT Knowledge Tracing Model. In *European Conference on Technology Enhanced Learning*. Springer, 183–197. <https://arxiv.org/pdf/2502.15709>
- [17] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, and Y. Piao. 2024. Deepseek-v3 Technical Report. *arXiv preprint arXiv:2412.19437* (2024). <https://arxiv.org/abs/2412.19437>
- [18] Guo X. Hu X. & Zhao X Liu, S. (2024). Advancing Generative Intelligent Tutoring Systems with GPT-4: Design, Evaluation, and a Modular Framework for Future Learning Platforms. *Electronics* ((2024)). <https://www.mdpi.com/2079-9292/13/24/4876>
- [19] LlamaIndex. 2025. Chat Engine - ReAct Agent Mode. https://docs.llamaindex.ai/en/stable/examples/chat_engine/chat_engine_react/ Last accessed 4 April 2025.
- [20] LlamaIndex. 2025. SimpleDirectoryReader. https://docs.llamaindex.ai/en/stable/module_guides/loading/simpledirectoryreader/ Last accessed 4 April 2025.
- [21] Kaushal Kumar Maurya, KV Aditya Srivatsa, Kseniia Petukhova, and Ekaterina Kochmar. 2024. Unifying AI Tutor Evaluation: An Evaluation Taxonomy for Pedagogical Ability Assessment of LLM-Powered AI Tutors. In *arXiv preprint arXiv:2412.09416*. <https://arxiv.org/abs/2412.09416>
- [22] A. Naya-Forcano, M. Garcia-Bosque, E. Cascarosa, F. Aznar, C. Sánchez-Azqueta, S. Celma, and C. Aldea. 2024. ChatPLT: An Intelligent Tutoring System for Teaching Physics in Higher Education. In *10th International Conference on Higher Education Advances (HEAd'24)*. Universitat Politècnica de València. doi:10.4995/HEAd24.2024.17261
- [23] Amita Rani, Komal Mehla, and Ajay Jangra. 2015. Parsers and parsing approaches: Classification and state of the art. In *2015 International Conference on Futuristic Trends on Computational Analysis and Knowledge Management (ABLAZE)*. 34–38. doi:10.1109/ABLAZE.2015.7154963
- [24] Replicate. 2025. Model: meta-llama-3-70b-instruct. <https://replicate.com/meta/meta-llama-3-70b-instruct> Last accessed 4 April 2025.
- [25] David Rozado. 2020. Wide range screening of algorithmic biases in word embedding models using large sentimentlexicons reveals under-reported bias types. <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0231189>
- [26] Kartik Sharma, Peeyush Kumar, and Yunqing Li. 2024. OG-RAG: Ontology-Grounded Retrieval-Augmented Generation For Large Language Models. *arXiv:2412.15235 [cs.CL]* <https://arxiv.org/abs/2412.15235>
- [27] Joseph Slade, Alina Hyk, and Regan Gurung. 2024. Transforming Learning: Assessing the Efficacy of a Retrieval-Augmented Generation System as a Tutor for Introductory Psychology. *Proceedings of the Human Factors and Ergonomics Society Annual Meeting* 68 (08 2024), 1827–1830. doi:10.1177/107111813241275509
- [28] A. Sticha. 2023. *Utilizing Large Language Models for Question Answering in Task-Oriented Dialogues*. Master's thesis. Department of Engineering, University of Cambridge.
- [29] Varun. 2020. Cosine similarity: How does it measure the similarity, Maths behind and usage in Python. <https://towardsdatascience.com/cosine-similarity-how-does-it-measure-the-similarity-maths-behind-and-usage-in-python-50ad30aad7db/> Last accessed 1 May 2025.
- [30] et al. von Dielingen. 2024. The Impact of an AI-Based Tutor Using GPT-4o Mini on Learning R Programming among Psychology Students at the University of Tübingen. (2024).
- [31] Rose E. Wang, Qingyang Zhang, Carly D. Robinson, Susanna Loeb, and Dorottya Demszky. 2024. Bridging the Novice-Expert Gap via Models of Decision-Making: A Case Study on Remediating Math Mistakes. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Association for Computational Linguistics, Mexico City, Mexico, 2174–2199. doi:10.18653/v1/2024.naacl-long.120
- [32] Hayden Wolff. 2023. Rag 101: Demystifying Retrieval-Augmented Generation Pipelines. <https://developer.nvidia.com/blog/rag-101-demystifying-retrieval-augmented-generation-pipelines/>